

06장 PySide6 심화 (PySide6 Advanced)

PySide6 심화 개요

PySide6로 창을 띄우고, 위젯을 배치하고, 시그널과 슬롯으로 사용자의 동작에 반응하는 방법까지 익혔습니다. 버튼을 누르면 무언가 일어나고, 입력한 값이 화면에 반영되는, 이른바 '작동하는 프로그램'을 만들 수 있게 되었습니다.

그런데 화면을 코드로 일일이 작성하는 일은 생각보다 고되고, 다뤄야 할 데이터가 많아질수록 위젯에 값을 직접 채워 넣는 방식은 금세 한계에 부딪히게 됩니다.

이번 장은 그 한계를 넘어서기 위한 도구들을 다룹니다. 화면 설계를 손으로 그리듯 시각적으로 할 수 있게 도와주는 방법, 코드와 화면을 분리해 관리하는 방식, 한 단계 더 성숙한 프로그램을 만들기 위한 사고방식과 기술을 폭넓게 살펴봅니다.

그리고 무거운 작업을 처리하면서도 화면이 멈추지 않게 하는 기법까지 짚어 봅니다. 이 모든 것은 결국 '돌아가는 프로그램'을 넘어 '잘 만들어진 프로그램'으로 나아가기 위한 과정입니다.

내용이 다소 어렵게 느껴질 수 있지만, 조급해할 필요는 없습니다. 앞 장에서 다진 기초가 있다면 충분히 따라올 수 있도록 차근차근 풀어 가겠습니다. 이 장을 마치고 나면 여러분은 혼자서도 제법 그럴듯한 데스크톱 애플리케이션을 설계하고 만들어 낼 수 있는 힘을 갖추게 될 것입니다.

목차

- [06-01 Qt Designer로 UI 디자인하기](#)
- [06-02 ui 파일 파이썬 코드로 변환](#)
- [06-03 모델/뷰 아키텍처](#)
- [06-04 세 가지 뷰와 모델](#)
- [06-05 그래픽 뷰 프레임워크](#)
- [06-06 스타일시트\(QSS\)로 꾸미기](#)
- [06-07 멀티스레딩과 QThread](#)

Qt Designer 활용

위젯을 생성하고 레이아웃 배치하는 과정을 코드로 진행해 왔습니다. 위젯이 몇 개 안 될 때는 괜찮지만, 화면이 복잡해질수록 코드만으로 배치가 점점 어려워집니다.

Qt Designer는 이런 화면 설계를 마우스로 할 수 있게 해 주는 도구입니다. 위젯을 끌어다 놓고, 속성을 표에서 바꾸고, 결과를 즉시 눈으로 확인합니다. 디자이너로 만든 화면은 '.ui' 파일로 저장되고, 이 파일을 파이썬 코드에서 불러와 사용합니다. 즉 "**화면 디자인은 Designer가, 동작은 파이썬이**" 말하는 식으로 역할을 나누는 것이 핵심입니다.

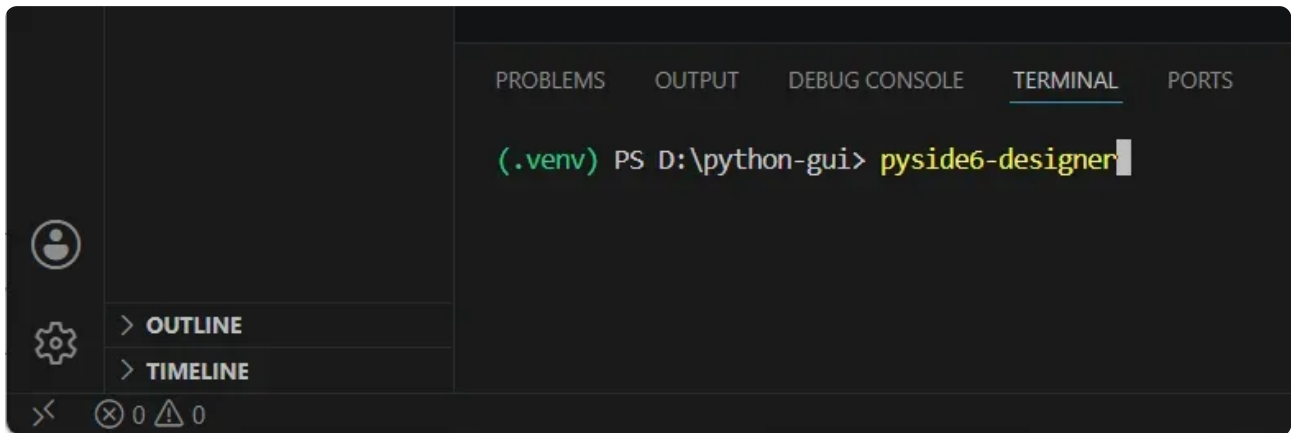
- [Qt Designer 실행과 화면 구성](#)
 - [실행 명령](#)
 - [새 폼\(New Form\) 선택하기](#)
 - [네 가지 주요 영역](#)
- [위젯 배치와 레이아웃 적용](#)
 - [새 폼 생성](#)
 - [위젯 배치와 속성 바꾸기](#)
 - [레이아웃 적용](#)
 - [Preview 사용하기](#)
- [.ui 파일로 저장하고 코드 실행](#)
 - [저장 방법과 파일 형식](#)
 - [.ui 파일](#)
 - [코드 실행](#)
 - [코드 한 줄씩 살펴보기](#)

Qt Designer 실행과 화면 구성

실행 명령

- PySide6를 설치하면 Qt Designer를 실행하는 명령이 함께 제공됩니다.
- 가상환경이 활성화된 상태에서 터미널에 다음을 입력합니다.

pyside6-designer



새 폼(New Form) 선택하기

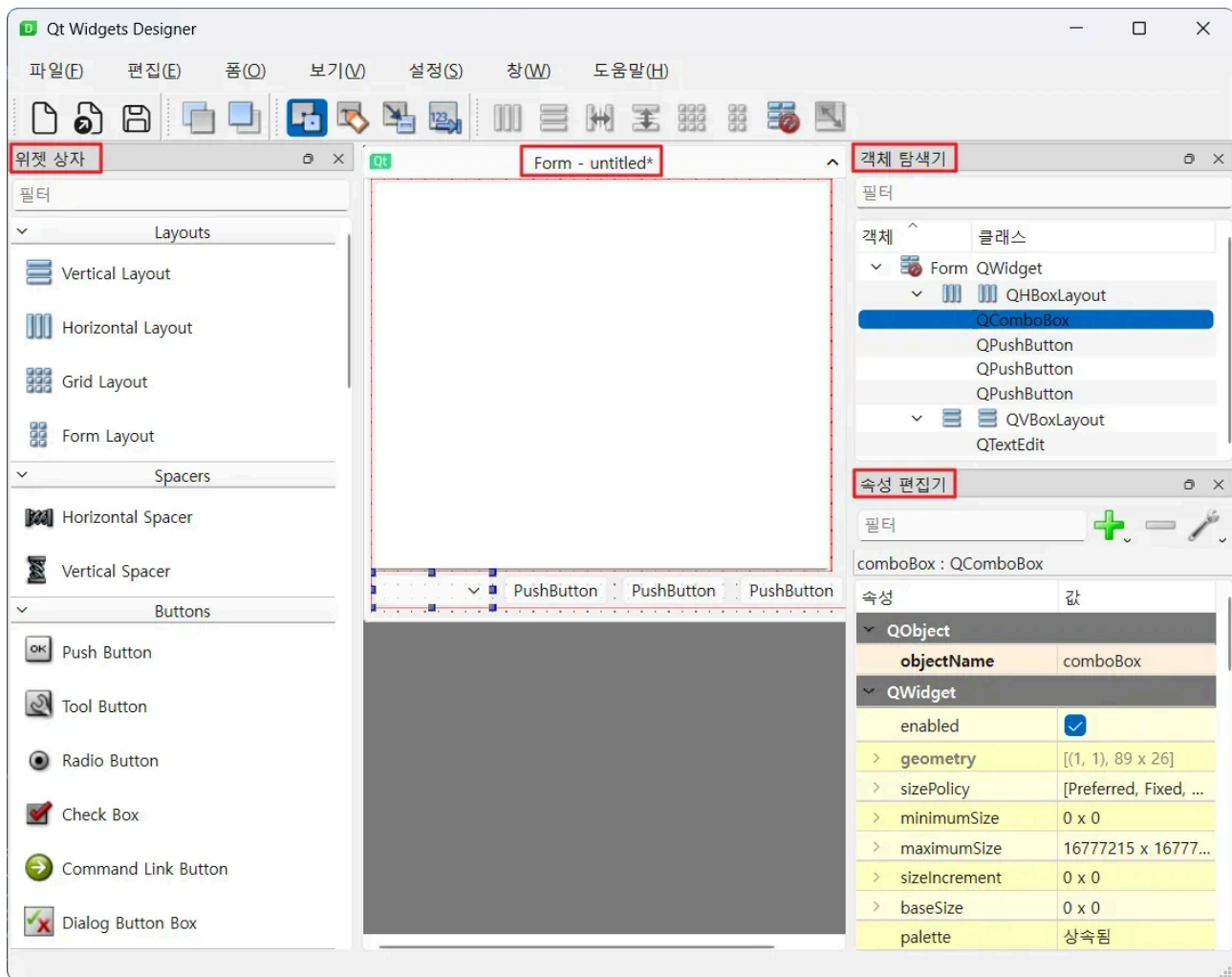
- 처음 실행하면 새 폼(New Form) 대화상자가 뜨는데, 어떤 종류의 화면을 만들지 고르는 단계입니다.

템플릿	설명
Main Window	메뉴·툴바·상태바를 갖춘 본격적인 애플리케이션 창 (<code>QMainWindow</code>)
Widget	메뉴 등이 없는 단순한 위젯 화면 (<code>QWidget</code>)
Dialog with Buttons Bottom	확인/취소 버튼이 아래에 달린 대화상자

네 가지 주요 영역

- 디자이너 창은 크게 네 영역으로 나뉩니다.

영역	이름	하는 일
왼쪽	위젯 상자(Widget Box)	버튼·라벨 등 사용할 수 있는 위젯 목록. 여기서 끌어다 놓는다
가운데	폼(Form)	실제로 화면을 꾸미는 작업 공간
오른쪽 위	객체 탐색기(Object Inspector)	화면에 올린 위젯들의 계층 구조를 트리로 보여준다
오른쪽 아래	속성 편집기(Property Editor)	선택한 위젯의 텍스트·크기·색 등 속성을 바꾼다



위젯 배치와 레이아웃 적용

- 실제 위젯을 배치하고 레이아웃을 적용하는 과정을 실습하며 개념 또한 익혀보겠습니다.

새 폼 생성

- 새 폼 에서 'Widget'를 선택하여 '생성'합니다.

위젯 배치와 속성 바꾸기

- 버튼 위젯 `Push Button` 을 폼에 끌어다 놓으면 "PushButton"이라는 글자가 적힌 버튼이 생깁니다.
- 이 버튼을 선택하고 속성 편집기에서 다음 두 가지를 바꿔 봅니다.

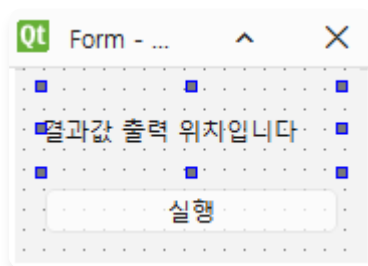
속성	의미	예시 값
<code>objectName</code>	코드에서 이 위젯을 가리킬 이름	<code>button</code>
<code>text</code>	화면에 보이는 글자	실행

`objectName` 은 특히 중요합니다. 나중에 파이썬 코드에서 이 위젯을 이름으로 찾아 쓰게 되므로, 알아보기 쉬운 이름을 붙여 두어야 합니다.

- Label 위젯도 폼에 끌어다 놓고 `objectName` 은 `label` 로 하고 `text` 에는 '결과값 출력 위치입니다.'로 설정합니다.

레이아웃 적용

- 앞서 코드로 `QVBoxLayout` , `QHBoxLayout` , `QGridLayout` 같은 레이아웃에 위젯을 배치해 봤는데, Designer에서는 같은 작업을 마우스만으로 할 수 있습니다.
- 독립형 Layout 위젯을 추가해서 앞서 생성한 '버튼'과 '라벨' 위젯을 자식으로 둘 수 있지만 여기서는 폼 자체(가장 바깥)에 레이아웃을 지정하겠습니다.
 - 폼의 빈 칸에 우클릭 '배치 → 수직으로 배치 Ctrl + 2'를 선택합니다.
 - 객체 탐색기에서 최상위 항목옆에 빨간색 금지 표시가 사라지는 것을 확인할 수 있습니다.



레이아웃을 적용하고 나면, 코드로 배치했을 때와 마찬가지로 창 크기를 늘리거나 줄여도 위젯들이 자연스럽게 재배치됩니다.

Preview 사용하기

- 화면이 어느 정도 갖춰지면 코드 작성 없이 Form → Preview(단축키 **Ctrl+R**)로 실행 결과를 바로 확인할 수 있습니다.
- 버튼을 눌러 보거나 창 크기를 조절해 레이아웃이 의도대로 동작하는지 점검하기 좋습니다.

.ui 파일로 저장하고 코드 실행

저장 방법과 파일 형식

- 작업이 끝나면 **File** → **Save** 로 파일이름을 'form.ui'로 저장합니다.
- 이때 확장자는 **.ui**이며, 그 내용은 **XML 형식의 텍스트**입니다.

.ui 파일

- 앞서 만든 버튼 하나와 라벨이 들어간 'form.ui' 파일은 대략 다음과 같은 모습입니다.

```
~ 생략 ~
<item>
  <widget class="QLabel" name="label">
    <property name="text">
      <string>결과값 출력 위치입니다</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QPushButton" name="button">
    <property name="text">
      <string>실행</string>
    </property>
  </widget>
</item>
~ 생략 ~
```

코드 실행

- 이제 'form.ui' 파일을 파이썬 코드에서 `QUiLoader()` 로 불러와 실제로 동작하는 프로그램으로 연결해 실행하는 코드입니다.

```
import sys
from PySide6.QtWidgets import QApplication
from PySide6.QtUiTools import QUiLoader
from PySide6.QtCore import QFile

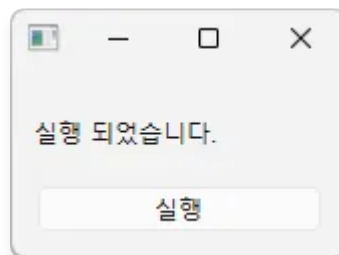
app = QApplication(sys.argv)

loader = QUiLoader()
ui_file = QFile("form.ui")
ui_file.open(QFile.OpenModeFlag.ReadOnly)
window = loader.load(ui_file)
ui_file.close()

def on_clicked():
    window.label.setText("실행 되었습니다.")

window.button.clicked.connect(on_clicked)

window.show()
sys.exit(app.exec())
```



코드 한 줄씩 살펴보기

- 로더 생성

```
loader = QUiLoader()
```


`.ui` 파일을 읽어 실제 위젯 객체로 만들어 주는 로더를 생성합니다. 이 객체가 XML로 기록된 화면 정의를 해석해 살아 있는 위젯 트리로 바꿔 줍니다.

- 파일 객체 생성

```
ui_file = QFile("form.ui")
```

디자이너로 저장한 'form.ui' 파일을 가리키는 파일 객체를 만듭니다. 아직 파일을 연 것은 아니며, 어떤 파일을 다룰지 지정만 한 단계입니다.

- 파일 열기

```
ui_file.open(QFile.OpenModeFlag.ReadOnly)
```

파일을 읽기 전용으로 엽니다. 로더는 내용을 읽기만 하므로 쓰기 권한은 필요하지 않습니다.

- 위젯으로 읽어서 window 돌려받기

```
window = loader.load(ui_file)
```

열린 파일을 로더에 넘겨 화면을 실제 위젯으로 만들고, 그 최상위 위젯을 `window` 로 돌려받습니다. 이후 `window.button`, `window.label` 처럼 `objectName` 으로 지정한 이름을 통해 각 위젯에 접근할 수 있습니다.

- 파일 닫기

```
ui_file.close()
```

로드가 끝났으므로 파일을 닫아 자원을 정리합니다. 위젯은 이미 메모리에 만들어졌으므로 파일을 닫아도 화면에는 영향이 없습니다.

ui 파일 파이썬 코드 변환

Qt Designer로 화면을 설계한 .ui 파일을 QUiLoader로 불러와 실행해 보았습니다. 이 방식은 변환 과정 없이 .ui 파일을 바로 읽어 들여 간편하지만, 위젯 이름이 코드에 드러나지 않아 자동완성의 도움을 받기 어렵고, 실행할 때마다 XML을 다시 해석해야 한다는 단점이 있습니다.

여기서는 실무에서 더 권장되는 방식을 살펴보겠습니다. pyside6-uic 명령으로 .ui 파일을 미리 파이썬 코드로 변환해 두고, 일반 모듈처럼 import해서 사용하는 사전 변환 방식입니다.

- [pyside6-uic로 변환하기](#)
 - [변환 명령](#)
 - [생성된 파일은 손대지 않는다](#)
 - [변환된 파일의 구조](#)
- [변환된 파일 사용하기](#)
 - [setupUi로 화면 불러오기](#)
 - [코드 한 줄씩 살펴보기](#)
 - [위젯에 접근하고 동작 연결](#)

pyside6-uic로 변환하기

변환 명령

- PySide6에는 `.ui` 파일을 파이썬 코드로 변환해 주는 `pyside6-uic` 명령이 들어 있습니다.
- 가상환경이 활성화된 터미널에서 다음과 같이 입력하면, `form.ui` 가 `ui_form.py` 라는 파이썬 파일로 변환됩니다.

```
pyside6-uic form.ui -o ui_form.py
```

`-o` 는 출력(output) 파일 이름을 지정하는 옵션입니다. 관례적으로 `ui_` 접두사를 붙여 "변환으로 생성된 파일"임을 표시합니다.

생성된 파일은 손대지 않는다

- 변환된 `ui_form.py` 는 자동 생성 파일이므로 **직접 수정하지 않습니다.**
- 디자인을 바꿀 때는 Designer에서 `.ui` 를 고친 뒤 변환 명령을 다시 실행합니다.

변환된 파일의 구조

- 변환된 파일 안에는 `Ui_` 로 시작하는 클래스가 하나 들어 있습니다.
- 클래스 이름은 Designer에서 지정한 최상위 객체 이름을 따릅니다. (예: 폼 이름이 `Form` 이면 `Ui_Form`)
- 이 클래스에는 `setupUi()` 라는 메서드가 있어, 호출하면 위젯들을 만들어 창에 배치해 줍니다.

```
class Ui_Form(object):
    def setupUi(self, Form):
        if not Form.setObjectName():
            Form.setObjectName(u"Form")
        Form.resize(168, 93)
        self.verticalLayout = QVBoxLayout(Form)
        self.verticalLayout.setObjectName(u"verticalLayout")
        self.label = QLabel(Form)
        self.label.setObjectName(u"label")
        # ... 이하 생략
```

변환된 파일 사용하기

setupUi로 화면 불러오기

- 변환된 `Ui_Form` 클래스를 사용합니다.
- 창으로 쓸 `QWidget` 을 하나 만든 뒤, `Ui_Form` 인스턴스의 `setupUi(window)`를 호출하면, Designer에서 만든 위젯들이 그 창(window)에 그대로 배치됩니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget
from ui_form import Ui_Form

app = QApplication(sys.argv)

window = QWidget()
ui = Ui_Form()
ui.setupUi(window)

window.show()
sys.exit(app.exec())
```

코드 한 줄씩 살펴보기

- 화면 정의 클래스 가져오기

```
from ui_form import Ui_Form
```

변환으로 생성된 `ui_form.py` 에서 화면 정의 클래스를 가져옵니다

- 인스턴스 생성

```
ui = Ui_Form()
```

화면 정의 클래스의 인스턴스를 만듭니다. 아직 화면에 그려진 것은 아니며, 배치를 담당할 객체를 준비한 단계입니다.

- 위젯 생성 및 배치

```
ui.setupUi(window)
```

위젯들을 실제로 생성·배치합니다. 이 호출이 끝나면 Designer에서 만든 화면이 그대로 나타납니다.

위젯에 접근하고 동작 연결

- `setupUi()`로 배치한 위젯은 `ui.위젯이름` 형태로 접근합니다. 위젯 이름은 Designer의 `objectName`으로 지정한 그 이름입니다.

```
import sys
from PySide6.QtWidgets import QApplication, QWidget
from ui_form import Ui_Form

def on_clicked():
    ui.label.setText("실행 되었습니다.")

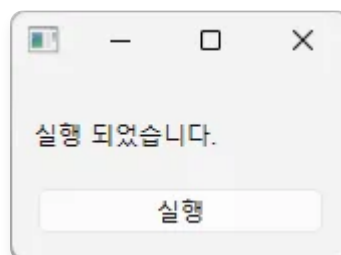
app = QApplication(sys.argv)

window = QWidget()
ui = Ui_Form()
ui.setupUi(window)

ui.button.clicked.connect(on_clicked)

window.show()
sys.exit(app.exec())
```

앞서 배운 시그널·슬롯이 여기서 그대로 쓰입니다. 화면은 **Designer**가 만들고, 동작 연결은 **코드**가 맡는 역할 분담, 곧 화면과 로직의 분리가 뚜렷이 드러납니다.



모델/뷰 아키텍처

위젯에 데이터를 직접 채워 넣는 아이템 기반 방식은 다루는 데이터가 적을 때는 직관적이고 편리합니다. 하지만 수백·수천 건의 목록이나 표를 다루기 시작하면 한계가 드러납니다. 데이터를 위젯에 하나하나 채워 넣다 보니 양이 많아질수록 느려지고, 데이터가 위젯 안에 갇혀 있어 같은 데이터를 표와 목록에 동시에 보여 주려면 양쪽을 따로 갱신해야 합니다.

Qt는 이 문제를 모델/뷰(Model/View) 아키텍처로 풀어냅니다. 데이터를 보관·관리하는 '모델'과 그것을 화면에 그리는 '뷰'를 분리하는 방식입니다. 여기서는 본격적인 코드에 들어가기 전에 모델/뷰가 왜 필요하고 어떤 구성으로 돌아가는지 그 큰 그림을 먼저 잡습니다.

- [모델/뷰가 필요한 이유](#)
 - [아이템 기반 이란](#)
 - [아이템 기반의 한계](#)
 - [모델/뷰의 장점](#)
- [세 가지 구성 요소](#)
 - [모델 \(Model\)](#)
 - [뷰\(View\)](#)
 - [델리게이트\(Delegate\)](#)
- [모델과 뷰의 통신 방식](#)
 - [인덱스로 데이터를 가리키기](#)
 - [역할\(Role\)로 용도를 구분하기](#)
 - [변경의 자동 반영](#)
- [모델 선택 기준](#)
 - [준비된 모델과 직접 만드는 모델](#)
 - [위젯 클래스와 뷰 클래스의 대응](#)
- [작은 예제](#)
 - [QStandardItemModel과 QListView 연결](#)
 - [코드 한 줄씩 살펴보기](#)
 - [하나의 모델, 두 개의 뷰](#)

모델/뷰가 필요한 이유

아이템 기반 이란

- 위젯에 값을 직접 넣는 방식을 **아이템 기반(item-based)** 이라고 부릅니다.
- `QListWidget` , `QTableWidget` , `QTreeWidget` 이 여기에 속하며, 항목 하나하나를 객체로 생성해 위젯에 추가합니다.

아이템 기반의 한계

- 항목마다 위젯 객체를 생성해야 하므로, 데이터가 많아질수록 메모리 소모가 크고 무거워집니다.
- 데이터가 UI(위젯)에 묶여 있어 따로 분리하거나 가공하기 번거롭습니다.

모델/뷰의 장점

- 하나의 데이터를 표로도 보여 주고 목록으로도 보여 주고 싶다면, 아이템 기반에서는 같은 내용을 두 위젯에 각각 채워 넣고, 데이터가 바뀔 때마다 양쪽을 모두 갱신해야 합니다.
- 모델/뷰에서는 데이터를 **모델 한 곳**에만 두고, 여러 뷰가 그 모델을 함께 바라봅니다. 데이터가 바뀌면 연결된 모든 뷰가 자동으로 갱신됩니다.

세 가지 구성 요소

모델 (Model)

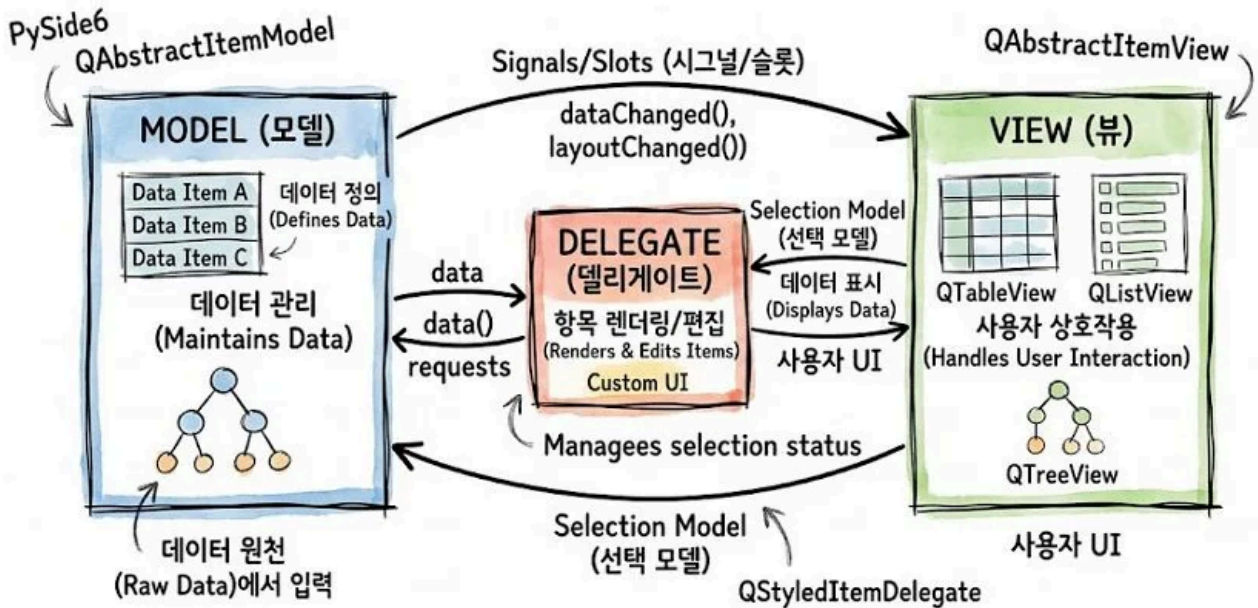
- 데이터를 보관하고 관리합니다.
- 데이터가 어떤 모습으로 화면에 그려질지는 신경 쓰지 않고, 오직 '무엇을 담고 있는가'만 책임집니다.

뷰(View)

- 모델이 가진 데이터를 화면에 그립니다.
- 데이터를 직접 들고 있지 않고, 필요할 때마다 모델에게 물어봐서 그립니다.
- 목록 모양의 `QListView` , 표 모양의 `QTableView` , 트리 모양의 `QTreeView` 가 대표적입니다.

델리게이트(Delegate)

- 각 항목이 실제로 어떻게 그려지고 어떻게 편집되는지를 담당합니다.
- 기본 델리게이트만으로도 충분한 경우가 많아, 입문 단계에서는 모델과 뷰에 집중하면 됩니다.



■ 알아두기

핵심은 **데이터(모델)**과 **표현(뷰)**의 분리입니다. tkinter에서 살펴본 MVP 패턴과 유사한 아키텍처가 Qt 차원에서 기본 제공되는 셈입니다.

모델은 화면을 몰라도 되고, 뷰는 데이터의 출처를 몰라도 됩니다. 덕분에 데이터 구조를 바꿔도 화면 코드를 건드릴 필요가 줄고, 화면을 바꿔도 데이터 코드는 그대로 둘 수 있습니다.

모델과 뷰의 통신 방식

인덱스로 데이터를 가리키기

- 모델 인덱스(`QModelIndex`)는 데이터의 한 칸을 행(row)과 열(column)로 지목하는 좌표입니다. 뷰는 이 인덱스로 모델에게 데이터를 요청합니다.
- 화면에 보이는 부분만 그때그때 물어보므로, 데이터가 많아도 효율적입니다.

역할(Role)로 용도를 구분하기

- 모델이 돌려주는 값에는 **역할(role)**이라는 꼬리표가 붙어, 같은 칸이라도 "표시할 글자"인지 "툴팁"인지 "글자 색"인지를 구분합니다.
- 가장 많이 쓰는 것은 표시할 텍스트를 뜻하는 `Qt.DisplayRole`입니다.

변경의 자동 반영

- 데이터가 바뀌면 모델이 신호를 보내고, 연결된 뷰들이 알아서 그 부분을 다시 그립니다.

- 그래서 개발자는 데이터만 바르게 관리하면 됩니다.

모델 선택 기준

준비된 모델과 직접 만드는 모델

- `QStandardItemModel` : 항목을 객체로 넣으면 알아서 행·열로 관리해 주는, 가장 다루기 쉬운 모델입니다. 처음 익힐 때 적합합니다.
- `QFileSystemModel` : 파일 시스템처럼 특정 용도에 맞춰진 모델도 있습니다.
- `QAbstractItemModel` 상속: 데이터가 특별한 구조이거나 외부에서 올 때 직접 만드는 방식입니다. 자유도는 가장 높지만 손이 많이 가므로, 보통 준비된 모델로 시작합니다.

위젯 클래스와 뷰 클래스의 대응

- 같은 화면을 데이터를 품은 채 쓰느냐(왼쪽), 모델과 분리해 쓰느냐(오른쪽)의 차이입니다.

화면 형태	아이템 기반 (데이터 포함)	모델/뷰 (데이터 분리)
목록	<code>QListWidget</code>	<code>QListView</code> + 모델
표	<code>QTableWidget</code>	<code>QTableView</code> + 모델
트리	<code>QTreeWidget</code>	<code>QTreeView</code> + 모델

- 항목이 적고 단순하면 아이템 기반이 간편합니다. 데이터가 많거나, 같은 데이터를 여러 화면에 보여 주거나, 데이터와 화면을 분리하고 싶을 때 모델/뷰가 빛을 발합니다.

작은 예제

`QStandardItemModel`과 `QListView` 연결

- 글로만 보면 추상적이니, 가장 단순한 형태로 모델과 뷰를 연결해 보겠습니다.
- 목록 데이터를 모델에 담고, 그 모델을 `QListView`에 물려 화면에 띄우는 예제입니다.

```

import sys
from PySide6.QtWidgets import QApplication, QListView
from PySide6.QtGui import QStandardItemModel, QStandardItem

app = QApplication(sys.argv)

model = QStandardItemModel()
for menu in ["갈비탕", "물냉면", "비빔냉면", "왕만두"]:
    model.appendRow(QStandardItem(menu))

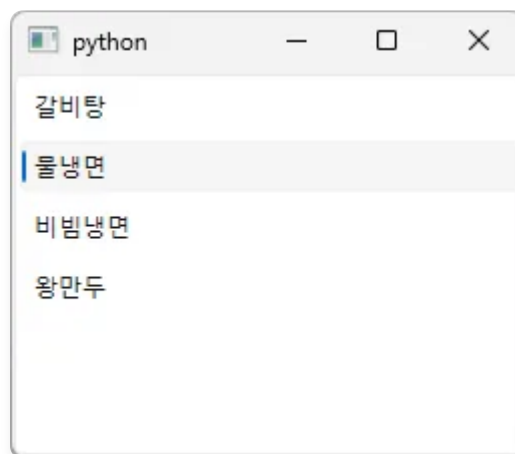
view = QListView()

view.setModel(model)

view.show()
sys.exit(app.exec())

```

코드의 흐름은 모델을 만들고 → 뷰를 만들고 → `setModel()` 로 둘을 잇는 세 단계입니다. 데이터를 위젯에 직접 넣지 않고 모델에만 담았다는 점, 그리고 뷰는 `setModel()` 한 줄로 그 데이터를 받아 그린다는 점이 핵심입니다.



코드 한 줄씩 살펴보기

- 빈 모델 생성

```
model = QStandardItemModel()
```

빈 모델을 하나 만듭니다. 아직 데이터도 뷰도 없는, 데이터를 담을 그릇입니다.

- 모델에 데이터 추가

```
model.appendRow(QStandardItem(menu))
```

메뉴 이름 하나를 `QStandardItem` 항목으로 감싸 모델에 한 행씩 추가합니다.

- 목록 형태의 뷰 생성

```
view = QListView()
```

이 시점의 뷰는 무엇을 그릴지 아직 모르는 빈 화면입니다.

- 뷰와 모델 잇기

```
view.setModel(model)
```

이 연결로 뷰는 모델에게 인덱스마다 `Qt.DisplayRole` 값을 물어 가며 항목을 그립니다.

하나의 모델, 두 개의 뷰

- 모델/뷰의 장점은 같은 모델을 두 뷰에 연결해 보면 한눈에 드러납니다. `view2 = QListView()` 를 하나 더 만들어 `view2.setModel(model)` 로 같은 모델을 물리면, 두 화면이 같은 데이터를 보여 주고 데이터를 바꾸면 함께 갱신됩니다.
- 아이템 기반이었다면 두 위젯에 같은 항목을 따로 채워야 했을 일이, 모델 하나를 공유하는 것으로 끝납니다.

세 가지 뷰와 모델

모델/뷰 아키텍처에서 **뷰(View)**는 모델의 데이터를 화면에 표시하고, 사용자의 선택·편집 같은 상호작용을 처리하는 위젯입니다. 핵심은 데이터를 보관하거나 제공하는 **모델**과 데이터를 보여 주는 **뷰**가 분리되어 있다는 점입니다. 덕분에 같은 모델도 목적에 따라 목록(List), 표(Table), 트리(Tree) 형태로 다르게 표시할 수 있습니다.

이 장에서는 PySide6에서 자주 사용하는 대표적인 세 가지 뷰 위젯인 QListView, QTableView, QTreeView의 특징과 기본 사용법을 살펴봅니다.

- [세 가지 뷰 개요](#)
 - [대표 뷰와 특징](#)
 - [공통 기반 클래스](#)
 - [공통 사용법](#)
 - [공통 주요 메서드](#)
 - [공통으로 자주 쓰는 옵션](#)
 - [공통 시그널](#)
- [QListView](#)
 - [목록 형태의 메뉴](#)
 - [주요 메서드](#)
 - [주요 옵션](#)
- [QTableView](#)
 - [표 형태 데이터 표시하기](#)
 - [주요 메서드](#)
- [QTreeView](#)
 - [계층 구조 데이터 표시하기](#)
 - [코드 한 줄씩 살펴보기](#)
 - [주요 메서드](#)

세 가지 뷰 개요

대표 뷰와 특징

- PySide6에서 자주 사용하는 모델/뷰 위젯은 다음 세 가지입니다. 데이터의 형태에 맞게 선택합니다.

뷰	표시 형태	주로 사용하는 모델
<code>QListView</code>	1차원 목록	<code>QStringListModel</code> , <code>QStandardItemModel</code>
<code>QTableView</code>	2차원 표(행/열)	<code>QStandardItemModel</code> , <code>QSqlTableModel</code>
<code>QTreeView</code>	계층(트리) 구조	<code>QStandardItemModel</code> , <code>QFileSystemModel</code>

공통 기반 클래스

- 세 위젯 모두 `QAbstractItemView` 를 상속하므로 선택 모드, 편집 트리거, 현재 인덱스, 공통 시그널 같은 기능을 비슷한 방식으로 사용합니다.
- 단, 헤더 설정은 모든 뷰의 공통 기능이 아닙니다. 헤더는 주로 `QTableView` 와 `QTreeView` 에서 다룹니다.

공통 사용법

- 세 뷰 모두 다음 흐름으로 사용합니다.

```
view = SomeView()      # 1. 뷰 생성
model = SomeModel()    # 2. 모델 생성
view.setModel(model)   # 3. 뷰와 모델 연결

view.show()
```

공통 주요 메서드

메서드	설명
<code>setModel(model)</code>	뷰에 모델을 연결
<code>model()</code>	현재 연결된 모델 반환
<code>currentIndex()</code>	현재 포커스/커서가 있는 항목의 인덱스 반환
<code>setCurrentIndex(index)</code>	특정 항목을 현재 항목으로 지정
<code>setSelectionMode(mode)</code>	선택 방식 지정
<code>setSelectionBehavior(behavior)</code>	선택 단위 지정: 항목, 행, 열
<code>setEditTriggers(triggers)</code>	편집 시작 조건 지정

공통으로 자주 쓰는 옵션

```
from PySide6.QtWidgets import QAbstractItemView

view.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers) # 읽기 전용
view.setSelectionMode(QAbstractItemView.SelectionMode.ExtendedSelection) # 여러 항목 선택
view.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows) # 행 단위 선택
```

옵션	대표 값	설명
<code>SelectionMode</code>	<code>SingleSelection</code> / <code>ExtendedSelection</code>	하나만 선택 / 여러 개 선택
<code>SelectionBehavior</code>	<code>SelectItems</code> / <code>SelectRows</code>	항목 단위 선택 / 행 단위 선택

공통 시그널

- `clicked`, `doubleClicked`, `activated` 시그널은 `QModelIndex` 를 전달하며, 이를 통해 선택 항목의 행·열·데이터를 확인할 수 있습니다.


```
def on_clicked(index):
    value = view.model().data(index)
    print(index.row(), index.column(), value)

view.clicked.connect(on_clicked)
```

QListView

- `QListView` 는 목록 데이터를 표시하는 뷰입니다.
- 기본 `ListMode` 에서는 한 행에 하나의 항목을 표시하고, `IconMode` 에서는 아이콘 보기처럼 배치할 수 있습니다.
- 간단한 문자열 목록은 `QStringListModel` 과 함께 사용하면 편리합니다.

목록 형태의 메뉴

- 문자열 리스트를 그대로 모델로 다룹니다.

```
import sys
from PySide6.QtWidgets import QApplication, QListView
from PySide6.QtCore import QStringListModel

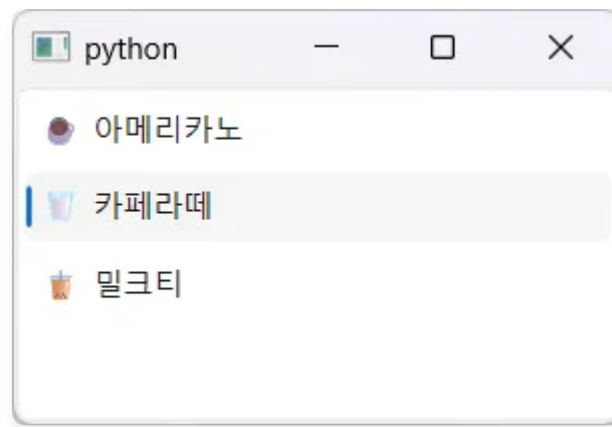
app = QApplication(sys.argv)

view = QListView()
model = QStringListModel(["☕ 아메리카노", "🥤 카페라떼", "🧋 밀크티"])

view.setModel(model)

view.clicked.connect(lambda index: print(f"주문 완료: {index.data()}"))

view.show()
sys.exit(app.exec())
```



주요 메서드

메서드	설명
<code>setViewMode(mode)</code>	목록 보기 또는 아이콘 보기 설정
<code>setFlow(flow)</code>	항목 배치 방향 설정
<code>setWrapping(bool)</code>	영역을 넘으면 줄바꿈할지 설정

주요 옵션

옵션	허용 값	설명
<code>ViewMode</code>	<code>ListMode</code> / <code>IconMode</code>	목록 형태 / 아이콘 격자 형태
<code>Flow</code>	<code>TopToBottom</code> / <code>LeftToRight</code>	세로 배치 / 가로 배치

QTableView

- `QTableView` 는 행과 열로 구성된 표 형태의 뷰입니다.
- 일반 데이터는 `QStandardItemModel` , 데이터베이스 테이블은 `QSqlTableModel` 과 함께 자주 사용됩니다.

표 형태 데이터 표시하기

- 행과 열로 이루어진 2차원 표를 화면에 표시합니다.

```

import sys
from PySide6.QtWidgets import QApplication, QTableView
from PySide6.QtGui import QStandardItemModel, QStandardItem

app = QApplication(sys.argv)

view = QTableView()

model = QStandardItemModel(3, 3)
model.setHorizontalHeaderLabels(["탐사선", "목적지", "현재 상태"])

data = [
    ["보이저 1호", "성간 우주", "통신 양호"],
    ["제임스 웹", "라그랑주 점 (L2)", "심우주 관측 중"],
    ["파커 솔라 프로브", "태양 코로나", "근일점 접근 중"],
]

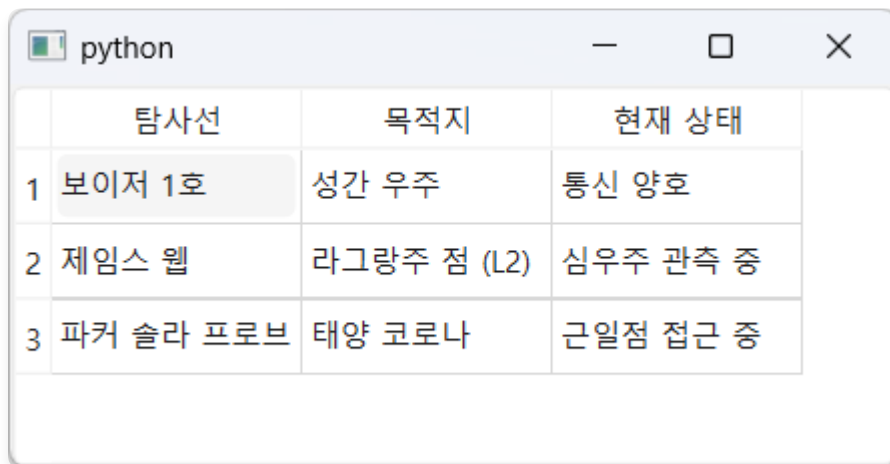
for row, record in enumerate(data):
    for col, value in enumerate(record):
        item = QStandardItem(value)
        model.setItem(row, col, item)

view.setModel(model)

view.show()

sys.exit(app.exec())

```



	탐사선	목적지	현재 상태
1	보이저 1호	성간 우주	통신 양호
2	제임스 웹	라그랑주 점 (L2)	심우주 관측 중
3	파커 솔라 프로브	태양 코로나	근일점 접근 중

주요 메서드

메서드	설명
<code>resizeColumnsToContents()</code>	열 너비를 내용에 맞게 자동 조절
<code>setColumnWidth(col, w)</code>	특정 열의 너비를 직접 지정
<code>setShowGrid(bool)</code>	격자선 표시 여부
<code>horizontalHeader()</code> / <code>verticalHeader()</code>	가로/세로 헤더 객체를 반환
<code>setSelectionBehavior(behavior)</code>	선택 단위(셀/행/열)를 지정

QTreeView

- `QTreeView` 는 부모-자식 관계가 있는 계층 데이터를 표시하는 뷰입니다.
- `QStandardItemModel` 을 사용할 때는 `QStandardItem` 에 자식 항목을 추가해 트리 구조를 만들 수 있습니다.

계층 구조 데이터 표시하기

```

import sys
from PySide6.QtWidgets import QApplication, QTreeView
from PySide6.QtGui import QStandardItemModel, QStandardItem

app = QApplication(sys.argv)

view = QTreeView()

model = QStandardItemModel()
model.setHorizontalHeaderLabels(["조직도"])
root = model.invisibleRootItem()

dev_div = QStandardItem("개발본부")
fe = QStandardItem("프론트엔드")
fe.appendRow(QStandardItem("이용"))

be = QStandardItem("백엔드")
be.appendRow(QStandardItem("김만수"))

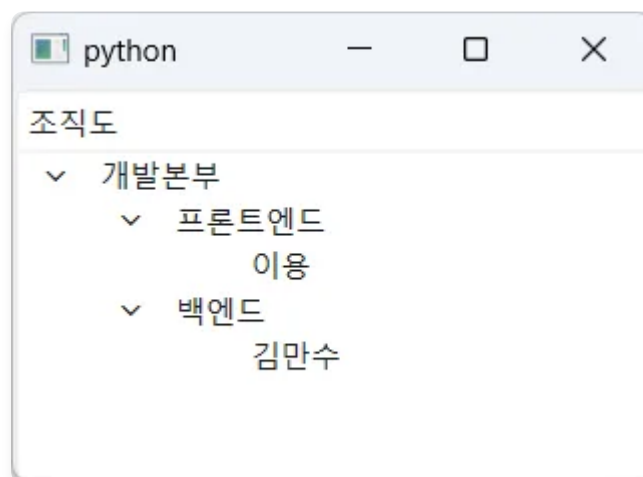
dev_div.appendRow(fe)
dev_div.appendRow(be)

root.appendRow(dev_div)

view.setModel(model)
view.show()

sys.exit(app.exec())

```



코드 한 줄씩 살펴보기

- 모델의 보이지 않는 최상위 루트 항목

```
root = model.invisibleRootItem()
```

실제 데이터 항목들은 모두 이 루트의 자식으로 붙습니다.

- `appendRow()` 로 자식 항목을 붙여 계층구조 생성

```
be = QStandardItem("백 엔드")
be.appendRow(QStandardItem("김 만 수"))
```

이렇게 부모-자식 관계를 만들면 `QTreeView` 가 이를 계층 구조로 표시합니다.

- 뷰와 모델을 연결

```
view.setModel(model)
```

모델의 데이터가 트리 화면에 표시됩니다. 모델/뷰 아키텍처에서 가장 중요한 연결 지점입니다.

주요 메서드

메서드	설명
<code>expandAll()</code>	모든 노드를 펼치기
<code>collapseAll()</code>	모든 노드 접기

그래픽 뷰 프레임워크

그래픽 뷰 프레임워크는 "도형·이미지·텍스트 같은 시각적 객체를 자유롭게 그리고 다루는" 구조입니다. PySide6에서 그래픽 뷰는 수천 개의 객체를 다뤄도 부드럽게 동작하도록 설계되어 있고, 각 도형을 마우스로 선택·이동·회전·확대할 수 있는 기능을 기본으로 갖추고 있습니다.

다이어그램 편집기, 노드 기반 에디터, 게임 보드, 이미지 주석 도구 같은 "객체를 배치하고 조작하는" 화면을 만들 때 바로 이 프레임워크를 사용합니다.

- [그래픽 뷰의 구조 이해하기](#)
 - [세 개의 핵심 클래스](#)
 - [세 요소의 관계](#)
- [기본 사용법](#)
 - [단순한 예제](#)
 - [도형 추가하기](#)
- [사용자 정의 항목 만들기](#)
 - [QGraphicsItem 상속하기](#)
 - [코드 한 줄씩 살펴보기](#)

그래픽 뷰의 구조 이해하기

세 개의 핵심 클래스

- **QGraphicsScene** 은 도형들이 올라가는 **무대(장면)** 입니다. 화면에 직접 보이지는 않고, 도형 객체들을 담고 좌표를 관리하는 논리적 공간입니다.
- **QGraphicsView** 는 그 장면을 들여다보는 **창(뷰)** 입니다. 하나의 장면을 여러 뷰가 서로 다른 각도·배율로 비출 수 있습니다. 스크롤과 확대/축소를 담당하는 위젯이 바로 이것입니다.
- **QGraphicsItem** 은 장면 위에 올라가는 **개별 도형**입니다. 사각형, 원, 텍스트, 이미지 등이 모두 이 항목(Item)으로 표현됩니다.

세 요소의 관계

- 정리하면 장면(Scene)이라는 무대 위에 항목(Item)들을 올려놓고, 뷰(View)라는 카메라로 비춰 본다고 이해하면 됩니다.

- 무대는 도형의 원본 좌표를 그대로 보관하고, 카메라는 그것을 어떤 배율·위치로 비출지만 결정하므로 둘은 완전히 분리되어 있습니다.

기본 사용법

단순한 예제

- 먼저 빈 장면을 만들고 뷰로 띄워 보겠습니다.

```
import sys
from PySide6.QtWidgets import QApplication, QGraphicsScene, QGraphicsView

app = QApplication(sys.argv)

scene = QGraphicsScene()
scene.setSceneRect(0, 0, 400, 300)

view = QGraphicsView(scene)
view.show()

sys.exit(app.exec())
```

- `setSceneRect()` 는 장면의 좌표 범위를 정합니다.
- Qt의 좌표는 화면과 마찬가지로 왼쪽 위가 (0, 0)이고 아래로 갈수록 y가 커집니다.
- 이 범위를 넘어서는 객체가 생기면 뷰에 스크롤바가 자동으로 나타납니다.

도형 추가하기

- 이제 무대 위에 도형을 올려 보겠습니다.
- `QGraphicsScene` 은 기본 도형을 손쉽게 추가하는 메서드를 제공합니다.


```

import sys
from PySide6.QtWidgets import QApplication, QGraphicsScene, QGraphicsView
from PySide6.QtGui import QPen, QBrush
from PySide6.QtCore import Qt

app = QApplication(sys.argv)

scene = QGraphicsScene()
scene.setSceneRect(0, 0, 400, 180)

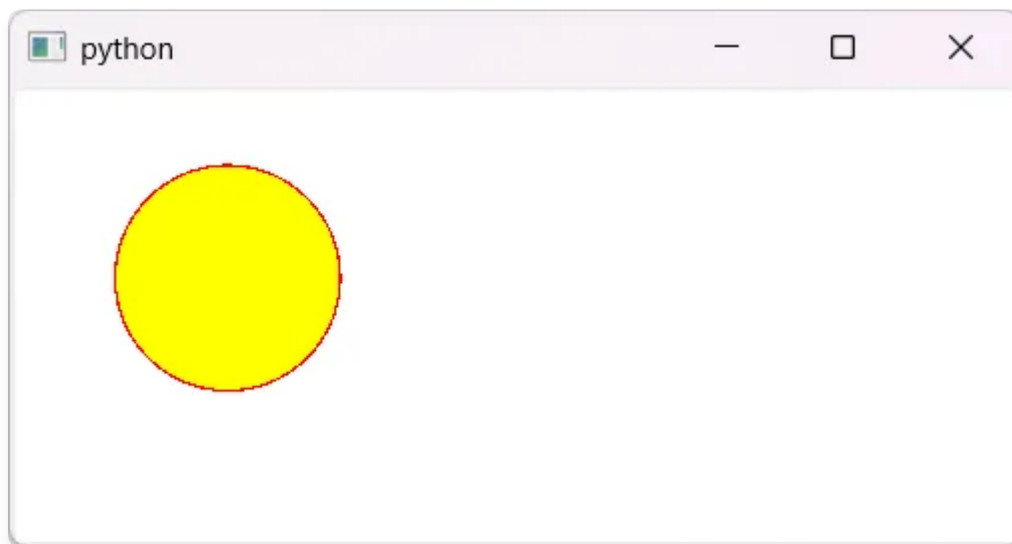
ellipse = scene.addEllipse(40, 30, 90, 90, QPen(Qt.red), QBrush(Qt.yellow))

view = QGraphicsView(scene)
view.show()

sys.exit(app.exec())

```

- `QPen` 은 도형의 테두리(선의 색과 두께), `QBrush` 는 채우기 색을 결정합니다.
- `addRect()` , `addEllipse()` , `addLine()` , `addText()` 같은 메서드는 각각 도형을 만들어 장면에 올린 뒤, 그 항목 객체를 돌려줍니다. 돌려받은 객체로 위치를 바꾸거나 속성을 조정할 수 있습니다.



사용자 정의 항목 만들기

QGraphicsItem 상속하기

- 기본 도형으로 부족할 때는 `QGraphicsItem` 을 상속해 **나만의 항목**을 정의할 수 있습니다.
- 아래 예제는 클릭하면 색이 바뀌고, 드래그로 옮길 수 있으며, 마우스 휠로 화면 전체를 확대·축소할 수 있는 상자를 만듭니다.

```

import sys
from PySide6.QtWidgets import (QApplication, QGraphicsScene, QGraphicsView,
                                QGraphicsRectItem, QGraphicsItem)
from PySide6.QtGui import QBrush, QColor

class ColorBox(QGraphicsRectItem):
    def __init__(self, x, y):
        super().__init__(0, 0, 80, 80)
        self.setPos(x, y)
        self.setBrush(QBrush(QColor("skyblue")))

        self.setFlags(QGraphicsItem.ItemIsMovable | QGraphicsItem.ItemIsSelectable)

    def mousePressEvent(self, event):
        new_color = "orange" if self.brush().color() == QColor("skyblue") else "skyblue"
        self.setBrush(QBrush(QColor(new_color)))
        super().mousePressEvent(event)

class ZoomView(QGraphicsView):
    def wheelEvent(self, event):
        self.scale(1.1, 1.1) if event.angleDelta().y() > 0 else self.scale(0.9, 0.9)

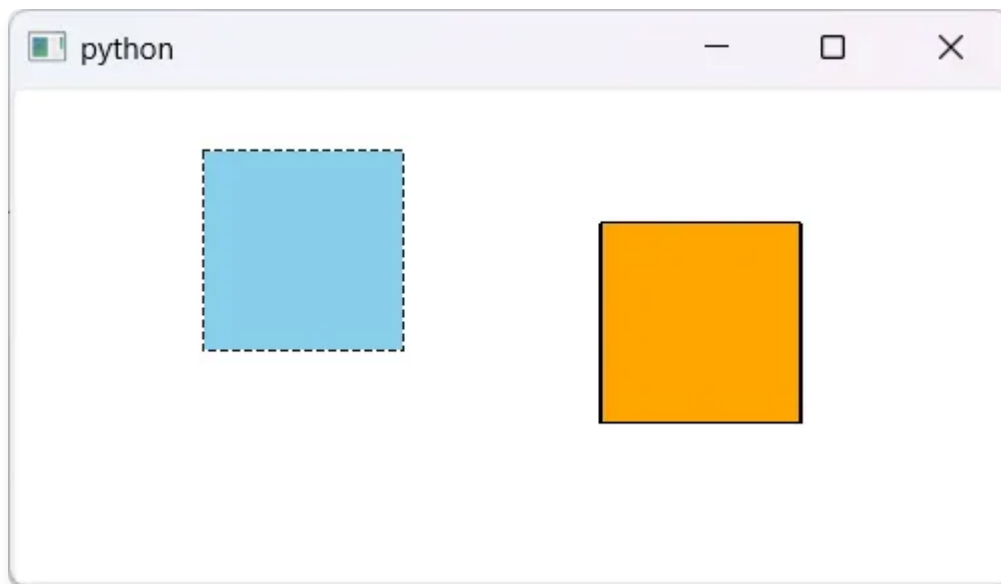
app = QApplication(sys.argv)
scene = QGraphicsScene(0, 0, 400, 300)

scene.addItem(ColorBox(50, 50))
scene.addItem(ColorBox(200, 150))

view = ZoomView(scene)
view.resize(450, 350)
view.show()

sys.exit(app.exec())

```



코드 한 줄씩 살펴보기

- 위젯 관련 클래스 가져오기

```
from PySide6.QtWidgets import (QApplication, QGraphicsScene, QGraphicsView,
                                QGraphicsRectItem, QGraphicsItem)
```

`QGraphicsScene` 은 도형이 놓이는 무대(논리적 공간), `QGraphicsView` 는 그 무대를 화면에 보여주는 창, `QGraphicsRectItem` 은 사각형 아이템, `QGraphicsItem` 은 아이템의 동작 플래그를 정의한 상위 클래스입니다.

- 사용자 정의 아이템 클래스 선언

```
class ColorBox(QGraphicsRectItem):
```

`QGraphicsRectItem` 을 상속한 사용자 정의 사각형 클래스 `ColorBox` 를 정의합니다.

- 아이템 초기화 및 위치·색 지정

```
def __init__(self, x, y):
    super().__init__(0, 0, 80, 80)
```

`super().__init__(0, 0, 80, 80)` 은 아이템 자체의 로컬 좌표계에서 (0, 0)을 기준으로 가로·세로 80 크기의 사각형을 만듭니다.

- 클릭 이벤트 재정의로 색 토글

```
def mousePressEvent(self, event):
    new_color = "orange" if self.brush().color() == QColor("skyblue") else "skyblue"
    self.setBrush(QBrush(QColor(new_color)))
    super().mousePressEvent(event)
```

아이템을 클릭하면 호출되는 `mousePressEvent` 를 재정의합니다. 현재 색이 하늘색이면 주황색으로, 아니면 하늘색으로 바꿔 클릭할 때마다 색이 번갈아 바뀝니다.

- 사용자 정의 뷰 클래스 선언

```
class ZoomView(QGraphicsView):
```

`QGraphicsView` 를 상속한 사용자 정의 뷰 `ZoomView` 를 정의합니다. 마우스 휠로 확대·축소 되는 창을 만들기 위함입니다.

- 휠 이벤트 재정의로 화면 확대·축소

```
def wheelEvent(self, event):
    self.scale(1.1, 1.1) if event.angleDelta().y() > 0 else self.scale(0.9, 0.9)
```

뷰 위에서 마우스 휠을 굴리면 호출되는 `wheelEvent` 를 재정의합니다. `event.angleDelta().y()` 가 양수이면(휠을 위로) 1.1배 확대하고, 음수이면(휠을 아래로) 0.9배 축소합니다.

- 장면 생성

```
scene = QGraphicsScene(0, 0, 400, 300)
```

`QGraphicsScene(0, 0, 400, 300)` 은 (0, 0)을 좌상단으로 하는 가로 400, 세로 300의 논리적 공간을 만듭니다.

- 아이템을 장면에 추가

```
scene.addItem(ColorBox(50, 50))
scene.addItem(ColorBox(200, 150))
```

`ColorBox` 두 개를 각각 (50, 50)과 (200, 150) 위치에 만들어 장면에 추가합니다. 아이템은 뷰가 아니라 **장면에 추가**한다는 점이 중요합니다. 장면이 아이템을 소유하고 관리하며, 뷰는 그 장면을 보여줄 뿐입니다.

- 뷰 생성.크기 지정.표시

```
view = ZoomView(scene)
```

`ZoomView(scene)` 로 앞서 만든 장면을 보여주는 뷰를 생성합니다. 하나의 장면을 여러 뷰가 동시에 볼 수도 있습니다.

스타일시트(QSS) 꾸미기

PySide6는 위젯의 겉모습을 손쉽게 바꿀 수 있도록 **스타일시트(QSS, Qt Style Sheets)** 기능을 제공합니다. 웹 개발의 CSS를 닮은 문법으로, 색·글꼴·여백·테두리 같은 시각 요소를 코드 한 줄 한 줄 그리지 않고도 선언적으로 지정할 수 있습니다.

- [QSS의 기본 개념](#)
 - [CSS를 닮은 스타일 언어](#)
 - [스타일을 적용하는 세 가지 범위](#)
- [기본 사용법](#)
 - [위젯 하나에 스타일 입히기](#)
 - [창 전체에 스타일 적용하기](#)
- [선택자로 대상 고르기](#)
 - [타입·ID·클래스 선택자](#)
 - [자주 쓰는 선택자 정리](#)
- [상태에 따라 스타일 바꾸기](#)
 - [의사 상태](#)
 - [하위 요소](#)
- [스타일시트를 파일로 관리하기](#)
 - [코드와 디자인 분리하기](#)
 - [파일을 읽어 적용하기](#)

QSS의 기본 개념

CSS를 닮은 스타일 언어

- QSS는 "어떤 위젯을, 어떻게 꾸밀지" 를 문자열로 적어 두는 방식입니다. 기본 형태는 다음과 같습니다.

```

선택자 {
    속성: 값;
    속성: 값;
}

```

- 예를 들어 모든 버튼의 배경을 파랗게 만들고 싶다면 이렇게 씁니다.

```

QPushButton {
    background-color: #3498db;
    color: white;
}

```

- `QPushButton` 이 선택자(어떤 위젯에 적용할지), 중괄호 안이 속성과 값(어떻게 꾸밀지)입니다.
- 웹의 CSS와 거의 같은 문법입니다.

스타일을 적용하는 세 가지 범위

- 스타일시트는 `setStyleSheet()` 메서드로 적용하며, 어느 객체에 호출하느냐에 따라 영향 범위가 달라집니다.
- `QApplication` 에 적용하면 프로그램 전체의 모든 위젯에 일괄 적용됩니다.
- 특정 위젯에 적용하면 그 위젯과 자식 위젯에만 적용됩니다.

```

app.setStyleSheet("QPushButton { color: red; }")    # 앱 전체 버튼
window.setStyleSheet("QPushButton { color: red; }") # 이 창 안의 버튼
button.setStyleSheet("color: red;")                 # 이 버튼 하나

```

기본 사용법

위젯 하나에 스타일 입히기

- 단순한 예제로, 버튼 하나의 모양을 바꿔 보겠습니다.


```
import sys
from PySide6.QtWidgets import QApplication, QPushButton

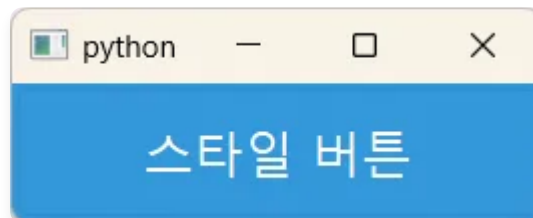
app = QApplication(sys.argv)

button = QPushButton("스타일 버튼")
button.setStyleSheet("""
    background-color: #3498db;
    color: white;
    font-size: 20px;
    padding: 7px;
""")

button.show()

sys.exit(app.exec())
```

위젯 하나에 직접 적용할 때는 선택자(`QPushButton { ... }`)를 생략하고 속성만 적어도 됩니다.



창 전체에 스타일 적용하기

- 여러 위젯이 있는 창에서는 선택자를 사용해 종류별로 한꺼번에 꾸미는 것이 편리합니다.

```

import sys
from PySide6.QtWidgets import (
    QApplication, QWidget, QVBoxLayout,
    QLabel, QLineEdit, QPushButton
)

app = QApplication(sys.argv)

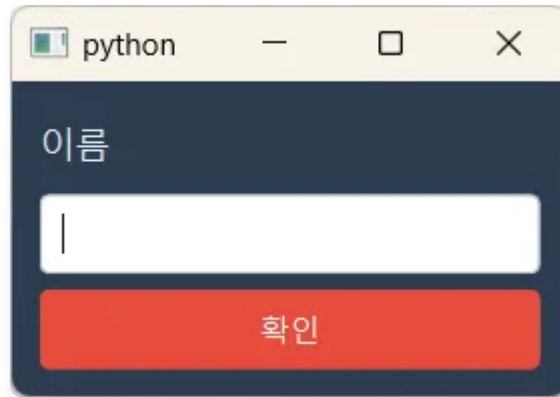
window = QWidget()
layout = QVBoxLayout()
layout.addWidget(QLabel("이름"))
layout.addWidget(QLineEdit())
layout.addWidget(QPushButton("확인"))
window.setLayout(layout)

window.setStyleSheet("""
    QWidget {
        background-color: #2c3e50;
    }
    QLabel {
        color: #ecf0f1;
        font-size: 14px;
    }
    QLineEdit {
        background-color: white;
        border: 1px solid #bdc3c7;
        border-radius: 4px;
        padding: 6px;
    }
    QPushButton {
        background-color: #e74c3c;
        color: white;
        padding: 8px;
        border-radius: 4px;
    }
""")

window.show()
sys.exit(app.exec())

```

창 전체에 한 번 적용해 두면 같은 종류의 위젯이 모두 같은 스타일을 따르므로, 일관된 화면을 손쉽게 유지할 수 있습니다.



선택자로 대상 고르기

타입·ID·클래스 선택자

- 선택자는 어떤 위젯을 고를지 결정하는 부분입니다. 같은 종류의 위젯이라도 일부만 다르게 꾸미고 싶을 때 선택자를 세분화합니다.
- 가장 기본은 위젯 종류로 고르는 **타입 선택자**(`QPushButton`)입니다.
- 특정 위젯 하나만 콕 집으려면 `setObjectName()` 으로 이름을 붙인 뒤 **ID 선택자**(`#이름`)로 고릅니다.

```
ok_button = QPushButton("확인")
ok_button.setObjectName("okButton")  # 이름 지정

window.setStyleSheet("""
    QPushButton {
        background-color: gray;        /* 모든 버튼 */
    }
    QPushButton#okButton {
        background-color: #27ae60;     /* 이름이 okButton인 버튼만 */
    }
""")
```

자주 쓰는 선택자 정리

선택자	의미	예시
<code>QPushButton</code>	해당 타입의 모든 위젯	<code>QPushButton { ... }</code>
<code>#okButton</code>	objectName이 okButton인 위젯	<code>#okButton { ... }</code>
<code>QPushButton#okButton</code>	타입 + 이름 함께 지정	더 명확하게 한정
<code>QDialog QPushButton</code>	QDialog 안에 있는 버튼	자손 관계로 한정
<code>.QPushButton</code>	해당 클래스(상속 제외)	정확히 그 타입만

- 이름이나 자손 관계로 한정할수록 더 구체적인 선택자가 되며, 구체적인 규칙이 일반 규칙보다 우선 적용됩니다.

상태에 따라 스타일 바꾸기

의사 상태

- 버튼은 평소 모습뿐 아니라 **마우스를 올렸을 때, 눌렀을 때, 비활성일 때** 모습이 다르면 훨씬 자연스럽습니다.
- QSS는 위젯의 상태를 **의사 상태**로 표현하며, 선택자 뒤에 콜론(`:`)으로 붙입니다.

```

import sys
from PySide6.QtWidgets import (
    QApplication, QWidget, QVBoxLayout, QPushButton
)

app = QApplication(sys.argv)

window = QWidget()

layout = QVBoxLayout()

active_button = QPushButton("마우스를 올려 보세요")
layout.addWidget(active_button)

disabled_button = QPushButton("비활성 버튼")
disabled_button.setEnabled(False)
layout.addWidget(disabled_button)

window.setLayout(layout)

window.setStyleSheet("""
    QPushButton {
        background-color: #3498db;
        color: white;
        font-size: 14px;
        padding: 8px;
        border: none;
        border-radius: 4px;
    }
    QPushButton:hover {
        background-color: #5dade2;
    }
    QPushButton:pressed {
        background-color: #2e86c1;
    }
    QPushButton:disabled {
        background-color: #bdc3c7;
    }
    """)

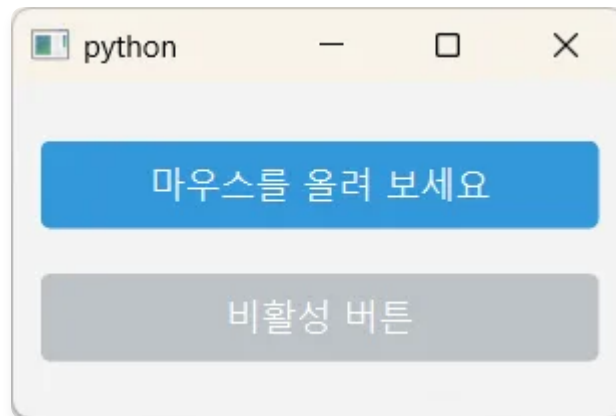
```

```

window.show()
sys.exit(app.exec())

```

이렇게만 적어 두면 사용자의 동작에 따라 알아서 해당 스타일로 전환해 줍니다. 별도의 이벤트 코드가 필요 없습니다.



하위 요소

- 체크박스의 네모 표시, 콤보박스의 화살표처럼 위젯을 이루는 부품도 따로 꾸밀 수 있습니다.
- 이를 하위 요소라 하며, 선택자 뒤에 더블 콜론(`::`)으로 붙입니다.

```

QCheckBox::indicator {
    width: 18px;
    height: 18px;
}
QComboBox::drop-down {
    border: none;
}

```

표기	이름	의미
<code>:hover</code>	의사 상태	위젯이 처한 상태를 가리킴 (콜론 하나)
<code>::indicator</code>	하위 요소	위젯을 이루는 부품을 가리킴 (콜론 둘)

콜론 하나는 "상태", 콜론 둘은 "부품"이라고 기억하면 헷갈리지 않습니다.

스타일시트를 파일로 관리하기

코드와 디자인 분리하기

- 스타일이 길어지면 스타일을 별도의 `.qss` 파일로 빼두고 읽어 들이면 코드와 디자인이 깔끔하게 분리됩니다.

```
/* style.qss */
QWidget {
    background-color: #2c3e50;
}
QPushButton {
    background-color: #e74c3c;
    color: white;
    padding: 8px;
    border-radius: 4px;
}
QPushButton:hover {
    background-color: #ec7063;
}
```

파일을 읽어 적용하기

- 작성한 `.qss` 파일은 일반 텍스트 파일이므로 파이썬으로 읽어 `setStyleSheet()` 에 넘기기만 하면 됩니다.

```
import sys
from PySide6.QtWidgets import QApplication, QPushButton

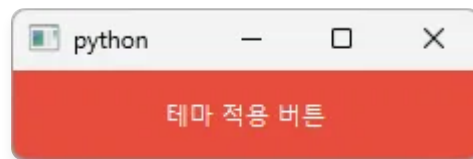
app = QApplication(sys.argv)

with open("style.qss", "r", encoding="utf-8") as f:
    app.setStyleSheet(f.read())

button = QPushButton("테마 적용 버튼")
button.show()

sys.exit(app.exec())
```

이렇게 하면 디자인을 수정할 때 파이썬 코드를 건드릴 필요 없이 `.qss` 파일만 고치면 되고, 같은 스타일 파일을 여러 프로그램에서 재사용할 수도 있습니다.



멀티스레딩과 QThread

대용량 데이터 처리나 파일 다운로드처럼 오래 걸리는 작업을 실행하면, 프로그램 창이 하얗게 굳으며 "응답 없음" 상태가 되곤 합니다.

이를 해결하는 핵심이 바로 스레드입니다. PySide6의 QThread를 사용해 무거운 작업을 백그라운드로 분리하면 GUI 멈춤을 방지할 수 있습니다. 이때 별도 스레드에서 처리된 결과를 메인 화면에 안전하게 전달하기 위해, 앞서 배운 시그널과 슬롯이 다시 한번 중요한 역할을 합니다.

이번 장에서는 화면이 멈추는 원인을 짚어보고, QThread를 활용한 작업 분리 및 시그널을 통한 안전한 데이터 통신 방법을 차근차근 학습합니다.

- [화면이 멈추는 이유](#)
 - [화면을 얼리는 무거운 작업](#)
 - [원인은 이벤트 루프의 중단](#)
- [QThread를 이용한 작업 분리](#)
 - [QThread 상속과 run\(\) 재정의](#)
 - [주요 메서드/시그널](#)
 - [다른 스레드의 위젯 직접 접근 금지](#)
- [시그널을 이용한 안전한 UI 업데이트](#)
 - [작업 스레드는 emit만, 화면 변경은 메인 스레드의 슬롯에서만](#)
 - [진행률을 표시하는 예제](#)
- [값 전달과 안전하게 멈추기](#)
 - [스레드에 값 전달](#)
 - [플래그를 이용한 안전한 중지](#)
- [Worker와 moveToThread 패턴](#)

화면이 멈추는 이유

화면을 얼리는 무거운 작업

- 버튼 클릭 시 일정시간 대기(`time.sleep`) 후 라벨을 변경하는 코드로 문제 상황을 재현해 보겠습니다.

```
import sys
import time
from PySide6.QtWidgets import (
    QApplication, QWidget, QLabel, QPushButton, QVBoxLayout
)

def heavy_task():
    for i in range(10):
        time.sleep(1)          # 무거운 작업 흉내
        label.setText("작업 완료!")

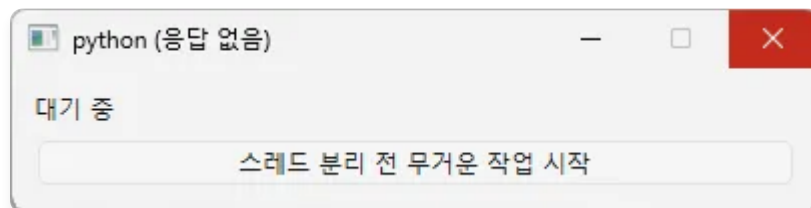
app = QApplication(sys.argv)
window = QWidget()

label = QLabel("대기 중")
button = QPushButton("스레드 분리 전 무거운 작업 시작")
button.clicked.connect(heavy_task)

layout = QVBoxLayout()
layout.addWidget(label)
layout.addWidget(button)
window.setLayout(layout)

window.show()
sys.exit(app.exec())
```

- 버튼을 눌러 보면 10초 동안 **창이 완전히 멈춥니다**. 창을 옮길 수도, 크기를 바꿀 수도 없어 "응답 없음" 표시가 뜨기도 합니다.



원인은 이벤트 루프의 중단

- `app.exec()` 가 실행하는 **이벤트 루프**는 마우스 클릭, 창 이동, 화면 갱신 등의 이벤트를 순서대로 처리합니다. 중요한 점은 이 모든 과정이 단 하나의 **메인 스레드**에서 이루어진다는 것입니다.
- 따라서 슬롯 함수 안에서 오래 걸리는 작업을 수행하면, 그동안 이벤트 루프가 막혀 다른 요청을 전혀 처리하지 못하게 됩니다.
- 이를 해결하려면 **무거운 작업을 별도의 스레드로 분리**하여 메인 스레드의 이벤트 루프가 계속 돌아갈 수 있도록 만들어야 합니다.

QThread를 이용한 작업 분리

QThread 상속과 run() 재정의

- `QThread` 를 상속받은 클래스를 만들고 `run()` 메서드를 재정의합니다. 이 `run()` 안에 작성한 코드가 별도의 스레드에서 실행됩니다.

⚠ 스레드를 실행할 때는 반드시 `start()` 메서드를 호출해야 합니다. `run()` 을 직접 호출하면 메인 스레드에서 실행되어 화면 멈춤이 그대로 발생합니다.

```
from PySide6.QtCore import QThread

class Worker(QThread):
    def run(self):
        # start()를 부르면 이 메서드가 새 스레드에서 실행
        for i in range(5):
            time.sleep(1)
```

주요 메서드/시그널

메서드/시그널	설명
<code>run()</code>	새 스레드에서 실행할 코드를 담는 곳. 재정의해서 사용
<code>start()</code>	스레드를 시작한다. 내부적으로 <code>run()</code> 을 호출
<code>wait()</code>	스레드가 끝날 때까지 기다린다
<code>isRunning()</code>	스레드가 실행 중인지 확인
<code>started</code>	스레드가 시작될 때 발생하는 내장 시그널
<code>finished</code>	<code>run()</code> 이 끝나 스레드가 종료될 때 발생하는 내장 시그널

- `started` 와 `finished` 는 `QThread` 가 기본 제공하는 시그널입니다. 특히 `finished` 는 "작업 끝!"을 알려주는 역할로 매우 자주 사용됩니다.

다른 스레드의 위젯 직접 접근 금지

- 가장 중요한 규칙입니다. `run()` 내부에서 라벨, 버튼 등 메인 스레드의 UI 위젯을 직접 조작해선 안 됩니다. 별도의 스레드가 GUI를 함부로 건드리면 프로그램이 예고 없이 강제 종료(Crash)될 수 있습니다.
- 그렇다면 작업 진행률이나 결과를 어떻게 화면에 반영할까요? 이때 필요한 것이 바로 시그널(Signal)입니다.

시그널을 이용한 안전한 UI 업데이트

작업 스레드는 emit만, 화면 변경은 메인 스레드의 슬롯에서만

- 작업 스레드가 화면을 직접 조작하면 안 됩니다. 대신 시그널을 발생(`emit`)시키면, 메인 스레드가 이를 받아 화면을 갱신합니다.
- 스레드 간 통신은 Qt가 알아서 안전하게 처리하므로, 위 원칙만 지키면 됩니다.

진행률을 표시하는 예제

- 시그널을 활용해 무거운 작업의 진행률을 UI로 안전하게 전달하는 예제입니다.

```

import sys
import time

from PySide6.QtWidgets import QApplication, QWidget, QPushButton, QVBoxLayout, QProgressBar
from PySide6.QtCore import QThread, Signal

class Worker(QThread):
    progress = Signal(int)

    def run(self):
        for i in range(1, 101):
            time.sleep(0.05)
            self.progress.emit(i)    # ㉠ 작업 스레드: 진행률 시그널 발생

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()

        self.progress_bar = QProgressBar()
        self.start_btn = QPushButton("스레드로 분리된 무거운 작업 시작")
        self.start_btn.clicked.connect(self.start_work)

        layout = QVBoxLayout(self)
        layout.addWidget(self.progress_bar)
        layout.addWidget(self.start_btn)

    def start_work(self):
        self.start_btn.setEnabled(False)
        self.worker = Worker()

        # ㉡ 메인 스레드: 시그널을 받아 UI 안전하게 갱신
        self.worker.progress.connect(self.progress_bar.setValue)
        self.worker.finished.connect(lambda: self.start_btn.setEnabled(True))

        self.worker.start()

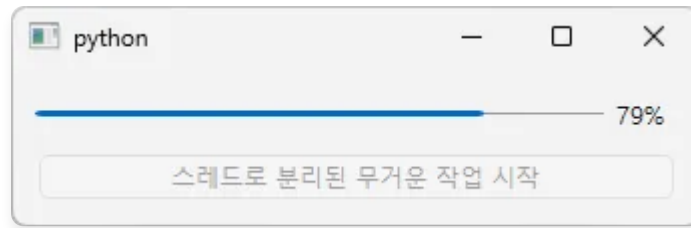
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()

```

```

window.show()
sys.exit(app.exec())

```



- 이번에는 작업이 도는 동안에도 창을 자유롭게 움직일 수 있고 진행률 막대가 부드럽게 차오릅니다. 메인 스레드의 이벤트 루프가 멈추지 않고 계속 돌기 때문입니다.

📌 알아두기

스레드 객체는 반드시 인스턴스 변수(`self.worker`)에 보관

- 위 예제에서 `self.worker = Worker()` 처럼 스레드 객체를 `self.` 에 저장해야 합니다.
- 만약 `worker = Worker()` 처럼 지역 변수로 두면, `start_work()` 가 끝나는 순간 객체가 사라져(가비지 컬렉션) 스레드가 제대로 동작하지 않거나 프로그램이 멈출 수 있습니다.

값 전달과 안전하게 멈추기

스레드에 값 전달

- 작업에 필요한 값(반복 횟수, 파일 경로 등)은 스레드를 생성할 때 `__init__` 으로 전달해 변수로 저장하고, `run()` 에서 꺼내 사용하는 것이 가장 깔끔합니다.

```

class Worker(QThread):
    progress = Signal(int)

    def __init__(self, total):
        super().__init__()
        self.total = total          # 작업에 필요한 값을 저장

    def run(self):
        for i in range(1, self.total + 1):
            time.sleep(0.5)
            self.progress.emit(int(i / self.total * 100))

```

- 사용할 때는 `self.worker = Worker(10)` 처럼 값을 넣어 생성합니다.

플래그를 이용한 안전한 중지

- 사용자가 "취소"를 눌렀다고 해서 스레드를 강제 종료하는 `terminate()` 를 쓰면 절대 안 됩니다. 작업이 어중간하게 끊겨 데이터가 망가지거나 프로그램이 충돌할 수 있습니다.
- 대신 상태를 나타내는 플래그(flag) 변수를 두어, `run()` 이 스스로 반복을 끝내고 자연스럽게 종료되도록 유도하는 것이 안전한 방식입니다.

```

import sys
import time
from PySide6.QtWidgets import (
    QApplication, QWidget, QLabel, QPushButton, QProgressBar, QVBoxLayout
)
from PySide6.QtCore import QThread, Signal

class Worker(QThread):
    progress = Signal(int)

    def __init__(self):
        super().__init__()
        self._running = True        # 작업을 계속할지 나타내는 플래그

    def run(self):
        count = 0
        while self._running and count < 100:
            time.sleep(0.1)
            count += 1
            self.progress.emit(count)

    def stop(self):
        self._running = False        # 플래그를 내려 run()의 반복을 끝낸다

class Window(QWidget):
    def __init__(self):
        super().__init__()

        self.bar = QProgressBar()
        self.start_btn = QPushButton("별도의 스레드 시작")
        self.stop_btn = QPushButton("플래그 이용한 중지")
        self.start_btn.clicked.connect(self.start_work)
        self.stop_btn.clicked.connect(self.stop_work)

        layout = QVBoxLayout()
        layout.addWidget(self.bar)
        layout.addWidget(self.start_btn)
        layout.addWidget(self.stop_btn)
        self.setLayout(layout)

```



```

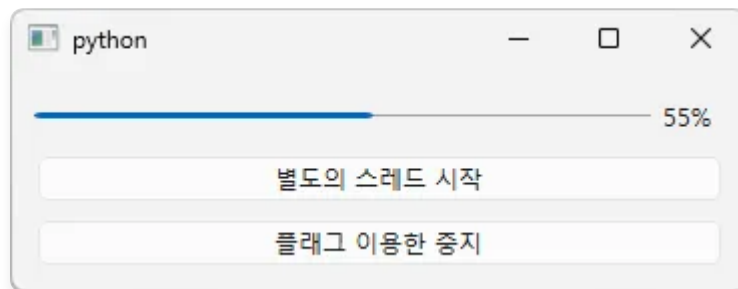
def start_work(self):
    self.worker = Worker()
    self.worker.progress.connect(self.bar.setValue)
    self.worker.start()

def stop_work(self):
    if hasattr(self, "worker") and self.worker.isRunning():
        self.worker.stop()      # 플래그를 내려 안전하게 종료 요청

app = QApplication(sys.argv)
window = Window()
window.show()
sys.exit(app.exec())

```

- "중지"를 누르면 `stop()` 이 플래그를 `False` 로 바꾸고, `run()` 의 `while` 조건이 거짓이 되어 반복이 자연스럽게 끝납니다.



Worker와 moveToThread 패턴

- 지금까지 배운 "`QThread` 를 상속해 `run()` 을 채우는" 방식은 이해하기 쉬워 입문 단계에 가장 적합합니다.
- 조금 더 권장하는 방식은 작업 내용은 일반 객체(`QObject`)에 담고, 그 객체를 `moveToThread()` 로 스레드에 옮기는 패턴입니다.

```

import sys
import time
from PySide6.QtCore import QObject, QThread, Signal
from PySide6.QtWidgets import QApplication, QWidget, QVBoxLayout, QPushButton, QProgressBar

class Worker(QObject):          # QThread가 아니라 QObject를 상속
    progress = Signal(int)
    finished = Signal()

    def run(self):
        for i in range(1, 51):
            time.sleep(0.05)
            self.progress.emit(i * 2)
        self.finished.emit()

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.pbar = QProgressBar(self)
        self.btn = QPushButton('moveToThread 시작', self)
        self.btn.clicked.connect(self.start_thread)

        vbox = QVBoxLayout()
        vbox.addWidget(self.pbar)
        vbox.addWidget(self.btn)
        self.setLayout(vbox)

        self.show()

    def start_thread(self):
        self.btn.setEnabled(False)
        self.pbar.setValue(0)

        self.thread = QThread()
        self.worker = Worker()

        self.worker.moveToThread(self.thread)          # 작업 객체를 스레드로 이동

```

```

self.thread.started.connect(self.worker.run) # 스레드가 시작되면 run 실행
self.worker.progress.connect(self.pbar.setValue)

self.worker.finished.connect(self.thread.quit)
self.worker.finished.connect(lambda: self.btn.setEnabled(True))

self.thread.start()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    main_win = MainWindow()
    sys.exit(app.exec())

```

- 이 방식은 작업(로직)과 스레드(실행 환경)를 분리해 구조가 더 유연하다는 장점이 있습니다. 다만 연결해야 할 시그널이 많아 처음에는 복잡하게 느껴질 수 있습니다. **지금 단계에서는 `QThread` 상속 방식으로 충분하며**, 이런 방식도 있다는 정도로 알아 두면 됩니다.

